

SPRING BOOT NOTES

What is Spring Boot?

- **Spring Boot** is a lightweight framework built on top of the Spring Framework to simplify and accelerate Java application development.
- It eliminates the need for complex **XML configuration** and provides an easy way to bootstrap and develop production-ready Spring applications with minimal code.
- Spring Boot follows the principle "**Convention over Configuration**", meaning developers get sensible defaults without needing to set up everything manually.
- It provides an environment to build **stand-alone, production-grade, and microservices-based** applications easily.

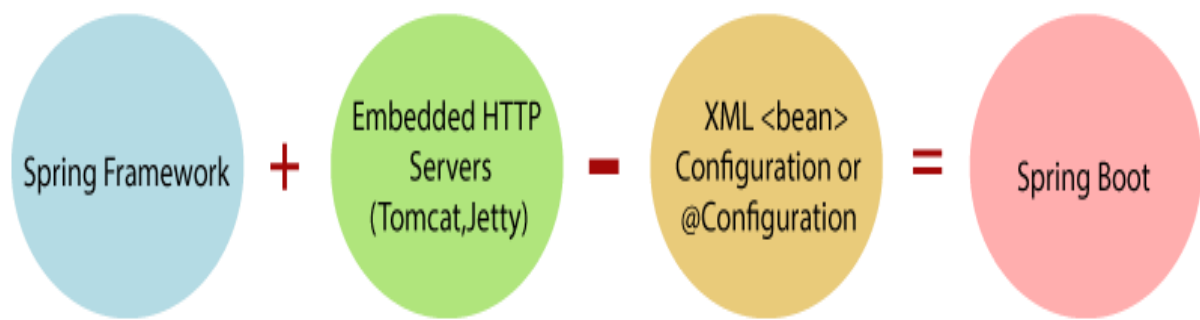
Key Components:

- **Spring Framework** – Core container and dependency injection features
- **Embedded HTTP Servers** – Tomcat, Jetty, Undertow (no need to install external server)
- **Auto-Configuration** – Automatically configures beans based on classpath dependencies
- **Starter Dependencies** – Bundled JARs that reduce dependency management effort

Spring Boot = Spring Framework + Auto Configuration + Embedded Servers + Starter Dependencies.

No need for **web.xml** or complex server setup. Spring Boot apps run directly using `java -jar`.

Developers commonly use **Spring Initializr** or **Spring STS** to generate Spring Boot project structure instantly.



Goals of Spring Boot

The main goal of Spring Boot is to **simplify Java development** and reduce the time spent on configuration.

- Provide a fast and opinionated development approach
- Reduce boilerplate code and avoid repetitive configurations
- Eliminate complex XML setup
- Offer production-ready features like metrics, health checks, and logging
- Make it easy to create **microservices** that run independently

Prerequisites for Spring Boot

To create and run a Spring Boot application, ensure you have the following:

- **Java 8+** (Java 17 recommended for modern Spring Boot versions)
- **Maven 3+** or **Gradle** for dependency management
- **Spring Framework knowledge** (IoC, DI, Beans)
- **IDE** such as IntelliJ IDEA, Spring Tool Suite, or Eclipse

Optional but recommended: Understanding of REST, JPA, and Microservices.

Spring Boot Features

Spring Boot provides a rich set of features that simplify application setup and development:

- **SpringApplication** – Bootstraps the entire application

- **Auto-Configuration** – Automatically configures Spring beans
- **Starter Dependencies** – Simplifies dependency management
- **Embedded Servers** – Tomcat, Jetty, Undertow
- **Externalized Configuration** – Manage settings using properties/YAML files
- **Logging** – Built-in Logback support with configurable log levels
- **YAML Support** – Cleaner configuration format
- **Actuator** – Production-ready monitoring endpoints
- **Security** – Easy integration with Spring Security for authentication
- **Developer Tools** – Auto restart, caching disablement during development

These features make Spring Boot ideal for building **enterprise applications, REST APIs,** and **cloud-native microservices.**

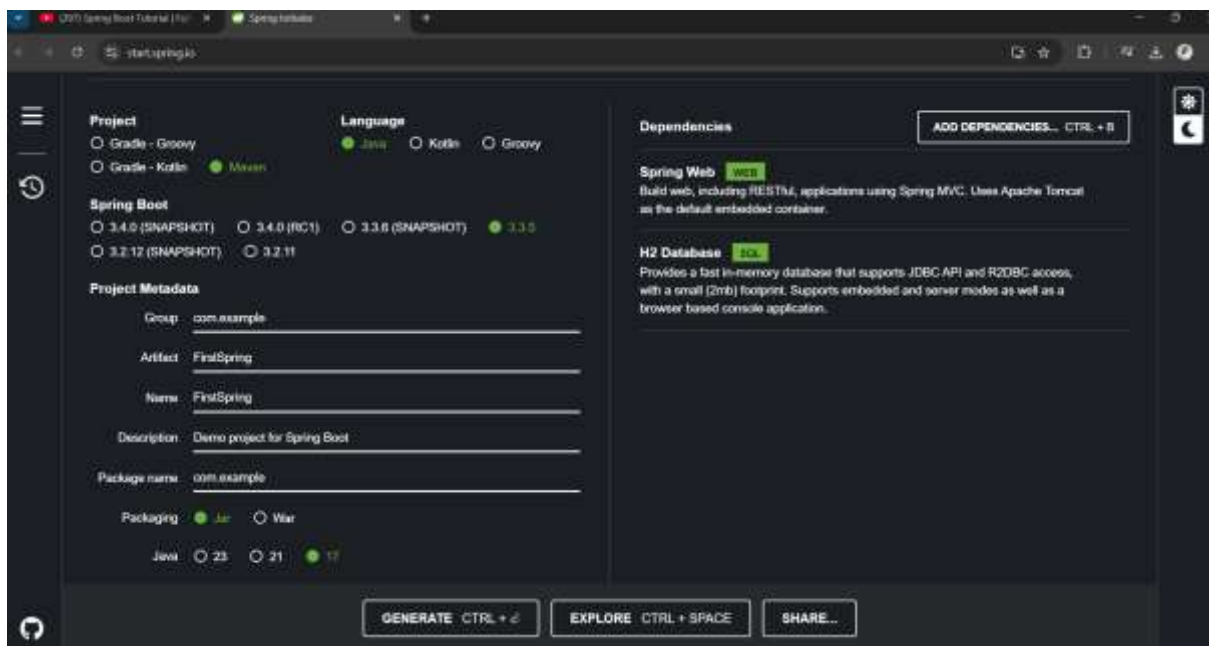
First Spring Boot Program

Step 1: Create a Spring Boot Project

You can create a Spring Boot project in two ways:

- **Spring Initializr** – Visit start.spring.io and generate a project with dependencies like Spring Web, JPA, MySQL, Lombok, etc.
- **Spring STS / IntelliJ / Eclipse** – Create a new Spring Starter Project directly from your IDE.

After generating the project, extract it and open it inside your preferred IDE.



Step 2: Main Application Class

The entry point of every Spring Boot application is a class annotated with `@SpringBootApplication`.

```
import org.springframework.boot.SpringApplication;

import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class MyApplication {

    public static void main(String[] args) {

        SpringApplication.run(MyApplication.class, args);

        System.out.println("Spring Boot Application
Started Successfully!");

    }

}
```

@SpringBootApplication is a meta-annotation that combines three powerful features:

- **@Configuration** – Marks the class as a configuration class where Spring Beans can be defined.
- **@EnableAutoConfiguration** – Enables Spring Boot's auto-configuration mechanism based on dependencies present in the classpath.
- **@ComponentScan** – Automatically scans the package (and sub-packages) for components like **@Controller**, **@Service**, **@Repository**, and **@Component**.

How the Program Works?

- **SpringApplication.run()** starts the Spring Boot application.
- It creates the Spring IoC container and initializes all beans.

- It starts the embedded server (Tomcat by default).
- Application becomes available at `http://localhost:8080` by default.

Important Notes:

- Spring Boot uses an **embedded server**, so no need to deploy WAR files manually.
- The project runs directly using:

```
mvn spring-boot:run
```

or

```
java -jar your-application.jar
```

- The default server port is **8080**, but it can be changed in `application.properties`:

```
server.port=9090
```

Spring Boot Architecture

- Spring Boot applications follow a **layered architecture**, where each layer has a specific responsibility.
- This structure ensures loose coupling, clean separation of concerns, and easier maintenance.

Spring Boot applications typically consist of the following **four layers**:

1. Presentation Layer

- Handles all incoming HTTP requests and outgoing HTTP responses
- Includes `@Controller`, `@RestController`
- Performs request validation, authentication entry, and response formatting

2. Business Layer

- Contains all business logic required by the application

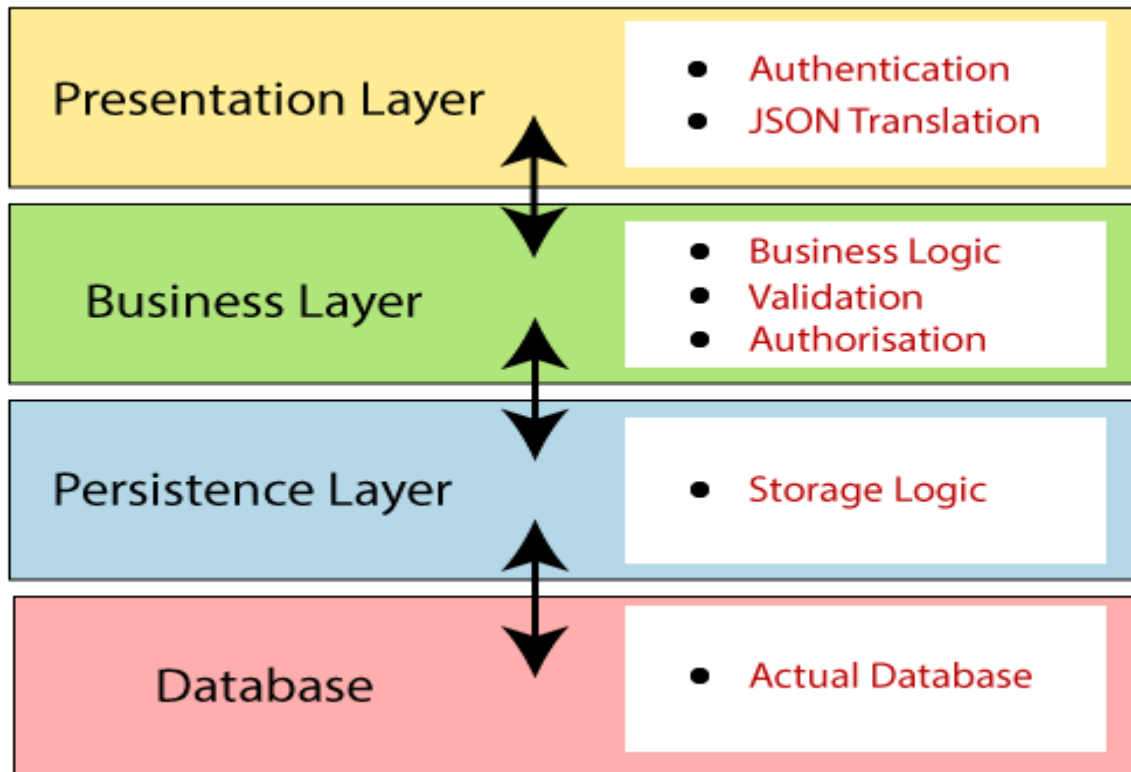
- Includes service classes annotated with `@Service`
- Handles validations, calculations, authorization, and business rules

3. Persistence Layer

- Interacts with the database via repositories
- Includes `@Repository` interfaces and JPA/Hibernate mappings
- Responsible for CRUD operations and data retrieval logic

4. Database Layer

- The physical database (MySQL, PostgreSQL, MongoDB, H2)
- Stores application data in tables/collections



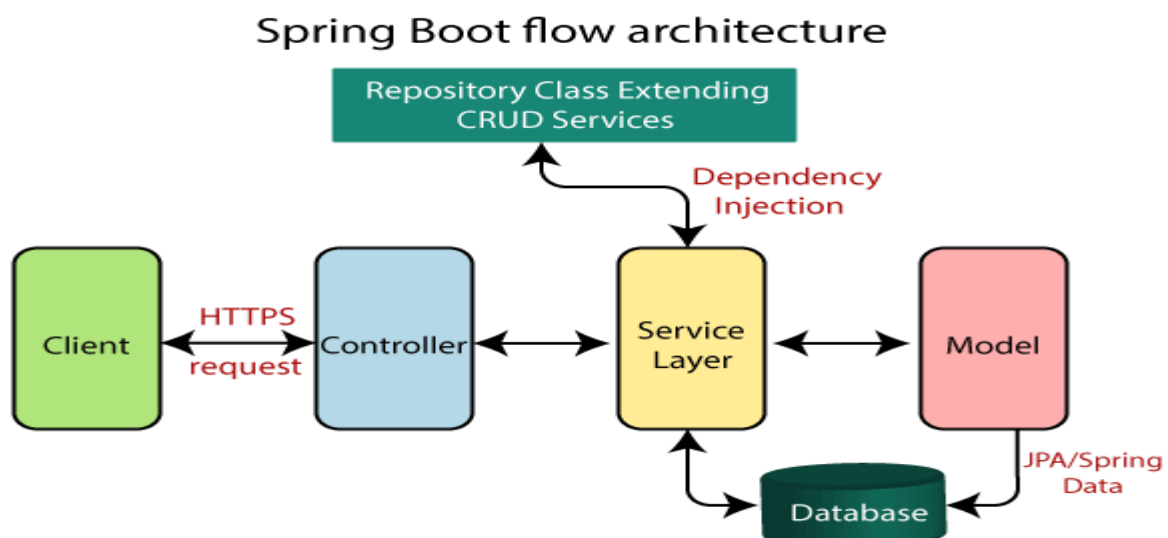
Spring Boot Request Flow

Below is the simplified flow of how a user request moves through the system:

- **Step 1:** The client sends an HTTP request (GET, POST, PUT, DELETE).
- **Step 2:** The request is handled by DispatcherServlet.
- **Step 3:** DispatcherServlet finds the appropriate controller via HandlerMapping.

- **Step 4:** Controller receives data and passes it to the service layer.
- **Step 5:** Service layer performs business logic and interacts with the repository.
- **Step 6:** Repository communicates with the database and returns the result.
- **Step 7:** Service returns processed results back to the controller.
- **Step 8:** Controller sends JSON/HTML response to the client.

This layered approach makes Spring Boot applications maintainable, testable, and scalable—ideal for microservice architecture.



Spring vs Spring Boot

Spring Boot is not a replacement for Spring — it is a simplified, faster, and developer-friendly extension of the Spring Framework.

Spring Framework	Spring Boot
Comprehensive Java development framework	Extension of Spring that simplifies configuration and development
Requires manual configuration of dependencies	Provides Spring Boot Starters to reduce setup complexity
Requires external server like Tomcat/Jetty	Includes embedded servers (Tomcat, Jetty, Undertow)
XML or Java-based configuration required	Auto-Configuration eliminates boilerplate setups

Spring Framework	Spring Boot
No built-in production monitoring	Provides Spring Boot Actuator (health, metrics, info)
Slower for developing microservices	Ideal for building microservices quickly

Spring Boot vs Spring MVC

Spring MVC is a web framework, while Spring Boot is a complete ecosystem for building production-ready apps.

Spring Boot	Spring MVC
Entire application framework with embedded server	Only the web layer of the larger Spring ecosystem
Auto-configures DispatcherServlet and other components	Requires manual setup of DispatcherServlet

Spring Boot	Spring MVC
Suitable for REST APIs and microservices	Focuses mainly on MVC-based web applications
Zero XML configuration	May require XML or Java configuration
Fast development with Spring Boot Starter dependencies	Developers must manage dependencies manually

Spring MVC can exist inside Spring Boot, but Spring Boot adds automation, simplicity, and production readiness on top of Spring MVC.

Inversion of Control (IoC) and Dependency Injection

What is the IoC Container?

Inversion of Control (IoC) is a design principle in which the control of object creation and dependency

management is transferred from the developer to the **Spring IoC container**.

The IoC container is responsible for:

- Creating and managing objects (called **beans**)
- Injecting required dependencies into objects automatically
- Managing the lifecycle of beans (creation → initialization → destruction)

Types of IoC Containers in Spring

- **BeanFactory** – Basic container with lazy initialization
- **ApplicationContext** – Advanced container with AOP, events, internationalization

What is Dependency Injection (DI)?

Dependency Injection (DI) is the technique used by Spring to implement IoC. Instead of creating objects

manually using new, the Spring container **injects** the required objects.

DI promotes:

- **Loose coupling**
- **Better testability**
- **Cleaner and maintainable code**

Types of DI in Spring

1. **Constructor Injection** – Best practice, recommended by Spring
2. **Setter Injection** – Allows optional dependencies
3. **Field Injection** (Not recommended)—used with `@Autowired` directly on fields

Benefits of DI

- Reduces tight coupling between components
- Simplifies unit testing (mocking becomes easier)

- Encourages cleaner and modular code
- Improves maintainability and scalability

Example: IoC & DI Mini Example

Without Spring (Tightly Coupled Code)

```
class Car {  
  
    Engine engine = new Engine(); // Manually creating  
    dependency  
  
}
```

This is tightly coupled because Car controls the creation of Engine.

With Spring IoC + DI (Loosely Coupled)

```
@Component  
  
class Engine {  
  
    public String start() {  
  
        return "Engine Started";  
    }  
}
```

```
}  
}
```

```
@Component
```

```
class Car {
```

```
    private final Engine engine;
```

```
    @Autowired // Constructor Injection
```

```
    public Car(Engine engine) {
```

```
        this.engine = engine;
```

```
    }
```

```
    public void drive() {
```

```
        System.out.println(engine.start());
```

```
    }
```